



SCS2312 - Assembly Language Tutorial

Tutorial Exercises

University of Colombo School of Computing

December 2025

Getting Started

- Write programs in NASM syntax.
- Compile: `nasm -f elf64 program.asm`
- Link: `ld program.o -o program`
- Run: `./program`

Note

Use Linux x86-64 environment. Online compilers or WSL/Ubuntu terminal can be used.

1 Exercise 1: Hello World

```
1  BITS 64
2  global _start
3
4  section .data
5      msg db "Hello, World!", 10
6      len equ $ - msg
7
8  section .text
9  _start:
10     mov rax, 1
11     mov rdi, 1
12     mov rsi, msg
13     mov rdx, len
14     syscall
15
16     mov rax, 60
17     xor rdi, rdi
18     syscall
```

Tip

Modify the message to print your own name.

2 Exercise 2: Add Two Numbers

Add 5 + 7 and print the result as a character.

- Store numbers: `mov rax, 5, mov rbx, 7`
- Add: `add rax, rbx`
- Convert to ASCII: `add rax, '0'`
- Print using write syscall

Note

Only single-digit results can be converted by adding '0'.

3 Exercise 3: Print Numbers 1-5

Use a loop to print numbers 1 through 5.

```
1 mov rcx, 1
2 loop_start:
3     ; convert rcx to ASCII
4     ; print character
5     inc rcx
6     cmp rcx, 6
7     jl loop_start
```

Tip

Print each number on a new line (add 10 for newline).

4 Exercise 4: Compare Two Numbers

Compare two numbers and print the larger one.

- Store numbers: `mov rax, 5, mov rbx, 7`
- Compare: `cmp rax, rbx`
- Conditional jump: `jg, jl`
- Print the larger number as ASCII

Warning

Always end the program with an exit syscall.

Submission Guidelines

- Submit all `.asm` files with clear file names.
- Include screenshots showing program output with comments.
- Write 1-2 lines explaining your approach for Exercise 4.
- Ensure code is readable, indented, and runs on NASM.
- Late submissions are not accepted.